

# DOCUMENTAÇÃO DE SOFTWARE

## Sistema Inteligente de Recomendação de Periféricos

### **Integrantes:**

- Arthur Felipe Rodrigues
- Arthur Cordeiro da Silva
- Felipe Henrique Ribeiro do Santos
- Igor França Candido
- Gabriel Francisco Alves da Silva

**Professor:** Mauro Rodrigues Alves Nogueira

**Disciplina:** Desenvolvimento Rápido de Aplicações em Python

### 1. Introdução

O presente projeto consiste no desenvolvimento de um software de recomendação de periféricos utilizando a linguagem Python.

O sistema foi desenvolvido com o objetivo de auxiliar usuários na escolha de periféricos com melhor custo-benefício, utilizando filtros inteligentes baseados em categoria e orçamento.

O software utiliza:

- Arquivos JSON para armazenamento de dados
- Interface gráfica interativa
- Sistema de recomendação por score
- Geração de gráficos estatísticos

### 2. Objetivo do Software

O objetivo principal do sistema é automatizar o processo de recomendação de periféricos, permitindo que o usuário encontre produtos adequados conforme sua necessidade e orçamento.

O sistema é capaz de:

- Carregar produtos automaticamente
- Filtrar periféricos por categoria
- Separar produtos por faixa de preço
- Calcular custo-benefício
- Recomendar os melhores produtos
- Gerar gráficos estatísticos

### 3. Tecnologias Utilizadas

| <b>Tecnologia</b> |  | <b>Função</b>                  |
|-------------------|--|--------------------------------|
| Python            |  | Linguagem principal do sistema |
| JSON              |  | Armazenamento dos dados        |
| ipywidgets        |  | Interface gráfica              |
| matplotlib        |  | Geração de gráficos            |
| Jupyter Notebook  |  | Ambiente de execução           |

### 4. Estrutura do Sistema

O software foi dividido em módulos para facilitar a organização e manutenção do código.

#### Estrutura dos módulos

#### 4.1 Módulo de Dados

Responsável pelo armazenamento dos produtos utilizando o arquivo produtos.json.

#### 4.2 Módulo de Filtragem

Responsável pela separação dos produtos conforme:

- categoria
- orçamento

#### 4.3 Módulo de Recomendação

Calcula o score dos produtos para identificar o melhor custo-benefício.

#### 4.4 Módulo de Interface

Responsável pela interação com o usuário através de menus e botões.

#### 4.5 Módulo Estatístico

Responsável pela geração de gráficos e médias.

### 5. Estrutura do Arquivo JSON

O sistema utiliza um arquivo chamado produtos.json.

Esse arquivo contém todos os periféricos cadastrados.

## Estrutura dos campos

| <b>Campo</b> | <b>Descrição</b>       |
|--------------|------------------------|
| nome         | Nome do produto        |
| preco        | Valor do produto       |
| dpi          | Sensibilidade do mouse |
| marca        | Marca do produto       |
| tipo         | Categoria do produto   |
| qualidade    | Nota de qualidade      |

## 6. Exemplo de Registro JSON

```
{
  "nome": "Mouse Pro RGB",
  "preco": 250,
  "dpi": 16000,
  "marca": "TopBrand",
  "tipo": "gamer",
  "qualidade": 9
}
```

## 7. Fluxo de Funcionamento do Sistema

O funcionamento do software ocorre nas seguintes etapas:

1. Inicialização do sistema
2. Leitura do arquivo JSON
3. Seleção do tipo de periférico
4. Seleção do orçamento
5. Filtragem dos produtos
6. Cálculo do score
7. Ordenação dos produtos
8. Exibição das recomendações
9. Geração do gráfico

## 8. Explicação Completa do Código

## 8.1 Importação das Bibliotecas

```
import json
import ipywidgets as widgets
from IPython.display import display, clear_output
import matplotlib.pyplot as plt
```

### Explicação

| Biblioteca   | Função                      |
|--------------|-----------------------------|
| json         | Manipulação do arquivo JSON |
| ipywidgets   | Interface gráfica           |
| display      | Exibição da interface       |
| clear_output | Limpeza da saída            |
| matplotlib   | Criação dos gráficos        |

## 9. Função carregar\_produtos()

```
def carregar_produtos():
    with open("produtos.json", "r") as f:
        return json.load(f)
```

### Objetivo

Responsável por abrir o arquivo JSON e carregar os produtos para dentro do sistema.

### Funcionamento

`open()`

Abre o arquivo `produtos.json`.

`"r"`

Modo leitura.

`json.load()`

Converte os dados JSON para estruturas do Python.

## 10. Função filtrar\_produtos()

```
def filtrar_produtos(produtos, tipo, orcamento):
```

### Objetivo

Filtrar os produtos conforme:

- tipo
- orçamento

#### Regras de orçamento

| <b>Categoria</b> | <b>Faixa</b>      |
|------------------|-------------------|
| baixo            | até R\$150        |
| medio            | R\$151 até R\$300 |
| alto             | acima de R\$300   |

#### Funcionamento

O sistema percorre todos os produtos utilizando:

```
for p in produtos:
```

Depois verifica:

- categoria
- preço

Se o produto atender aos critérios, ele é adicionado na lista:

```
filtrados.append(p)
```

## 11. Sistema de Recomendação

Função `calcular_score()`

```
def calcular_score(produto):
```

#### Objetivo

Calcular a pontuação dos produtos.

#### Fórmula Utilizada

[

Score = (10 - (preço / 50)) + (qualidade \times 2)

]

### Explicação da Fórmula

#### Pontuação do preço

```
preco_score = 10 - (produto["preco"] / 50)
```

Produtos mais baratos recebem mais pontos.

#### Pontuação da qualidade

```
qualidade_score = produto["qualidade"] * 2
```

Produtos com maior qualidade recebem maior pontuação.

### Resultado Final

```
return preco_score + qualidade_score
```

A soma define o custo-benefício do produto.

## 12. Função recomendar()

```
def recomendar(produtos):  
    return sorted(produtos, key=calcular_score, reverse=True)
```

### Objetivo

Ordenar os produtos do melhor para o pior score.

### sorted()

A função `sorted()` organiza automaticamente os produtos.

### reverse=True

Faz com que os maiores scores apareçam primeiro.

### 13. Interface Gráfica

O sistema utiliza ipywidgets para criar uma interface simples.

#### Dropdown de categoria

```
tipo_widget = widgets.Dropdown(  
    options=['gamer', 'escritorio']  
)
```

Permite selecionar:

- gamer
- escritório

#### Dropdown de orçamento

```
orcamento_widget = widgets.Dropdown(  
    options=['baixo', 'medio', 'alto']  
)
```

Permite selecionar:

- baixo
- médio
- alto

#### Botão de recomendação

```
button = widgets.Button(description="Recomendar")
```

Responsável por iniciar o sistema de recomendação.

### 14. Evento do Botão

```
button.on_click(on_button_clicked)
```

Quando o botão é pressionado:

- os produtos são filtrados
- o score é calculado

- as recomendações são exibidas
- o gráfico é gerado

### 15. Cálculo da Média de Preços

```
media = sum(p["preco"] for p in filtrados) / len(filtrados)
```

#### Objetivo

Calcular a média de preço dos produtos encontrados.

### 16. Sistema Estatístico

O software cria médias por categoria:

- baixo
- médio
- alto

Essas médias são utilizadas na geração dos gráficos.

### 17. Geração de Gráficos

```
plt.bar(labels, medias)
```

#### Objetivo

Criar um gráfico de barras contendo:

- categorias
- médias de preço

#### Configurações do gráfico

##### Título

```
plt.title("Média de Preços por Categoria")
```

##### Eixo X

```
plt.xlabel("Categoria de Orçamento")
```

Eixo Y

```
plt.ylabel("Preço Médio (R$)")
```

## 18. Requisitos Funcionais

| Código | Descrição           |
|--------|---------------------|
| RF01   | Carregar produtos   |
| RF02   | Filtrar produtos    |
| RF03   | Calcular score      |
| RF04   | Gerar recomendações |
| RF05   | Exibir gráficos     |

## 19. Requisitos Não Funcionais

| Código | Descrição               |
|--------|-------------------------|
| RNF01  | Interface simples       |
| RNF02  | Fácil manutenção        |
| RNF03  | Execução rápida         |
| RNF04  | Compatível com Python 3 |

## 20. Execução do Sistema

### Passos para execução

1. Instalar Python
2. Instalar bibliotecas:
  - a. matplotlib
  - b. ipywidgets
3. Criar o arquivo produtos.json
4. Executar o código Python
5. Selecionar as opções desejadas

## 21. Resultados Obtidos

O software conseguiu:

- recomendar produtos automaticamente
- organizar produtos por custo-benefício
- gerar gráficos estatísticos
- facilitar a visualização dos dados

## 22. Conclusão

O desenvolvimento deste projeto permitiu aplicar diversos conceitos importantes de desenvolvimento de software utilizando Python.

Entre os principais conceitos utilizados estão:

- manipulação de arquivos JSON
- estruturas condicionais
- funções
- algoritmos de recomendação
- interfaces gráficas
- geração de gráficos

O sistema demonstrou como técnicas simples de programação podem ser utilizadas para criar soluções inteligentes e organizadas.